

Speed Complete Notes

📅 Date	August 6, 2026
--------	----------------

Performance Fix Notes

1. Rule catalog cache extended (5 min → 1 hour)

python

```
@st.cache_data(ttl=3600)    # was ttl=300
def get_cached_rule_catalog():
```

The catalog holds 88,391 SIDs parsed from JSON. At `ttl=300`, this full parse re-ran every 5 minutes mid-session regardless of whether you were even using it — causing periodic 1.5–2.7s spikes unrelated to anything you clicked. Since the file doesn't change during a live session, there's no reason to expire it that often.

2. Ingestion moved to a background thread

python

```
def _background_ingest():
    try:
        ingest_logs()
    except Exception as e:
        print(f"[ERROR] Background ingestion failed:{e}")

def _maybe_ingest():
    now = time.time()
    last = st.session_state.get("_last_ingest_time", 0)
    if now - last >= INGEST_INTERVAL_SECONDS:
        st.session_state["_last_ingest_time"] = now
        threading.Thread(target=_background_ingest, daemon=True).start()
```

Previously, ingestion ran synchronously inline — every 15 seconds, the UI thread itself was blocked while `eve.json` was read and rows were written to SQLite. Now it fires off in a separate thread and the page keeps rendering immediately without waiting for that work to finish. This is why you saw `ingest=0.00s` in the timing logs afterward — it no longer counts against render time at all, it's just running independently in the background. The `missing ScriptRunContext` warning in your log is Streamlit noting this thread isn't part of its normal tracking — expected and explicitly harmless.

3. SQLite connection wrapped in a context manager

python

```
@contextmanager
def get_db_connection():
    conn = sqlite3.connect("alerts.db", timeout=10.0)
    conn.execute("PRAGMA journal_mode=WAL;")
    try:
        yield conn
    finally:
        conn.close()
```

Mostly a cleanliness/reliability improvement — guarantees the connection always closes even if an exception happens mid-query, using `with get_db_connection() as conn:` instead of manual try/finally blocks repeated in each function.

4. Raw JSON parsing simplified — dropped the recursive sanitizer

python

```
def _get_parsed_raw(row):
    ...
    return json.loads(x)  # was: sanitize_json_data(json.loads(x))
```

`sanitize_json_data` used to walk every key/value in the raw event recursively, applying regex emoji-stripping to every string field, every time a row was clicked.

That's now skipped entirely for the raw JSON view — it's shown as-is. (Note: this means any emoji/garbled characters in raw event fields will now show up unfiltered in the "Full raw event" JSON viewer — a real tradeoff for speed, not necessarily something you want long-term if the raw view still looks messy.)

5. Regex patterns compiled once at module load, not on every call

python

```
EMOJI_PATTERN = re.compile(...)
DASH_PATTERN = re.compile(r"\s*-\s*-\s*")
WHITESPACE_PATTERN = re.compile(r"\s+")

def sanitize_rule_text(text):
    cleaned = EMOJI_PATTERN.sub("", str(text))
    cleaned = DASH_PATTERN.sub(" - ", cleaned)
    cleaned = WHITESPACE_PATTERN.sub(" ", cleaned)
    return cleaned.strip(" -")
```

Previously, `re.compile(...)` ran fresh inside `sanitize_rule_text` every single time it was called — once per signature, per row, per render. Compiling once at import time and reusing the compiled pattern objects is a small but real win, especially since this function runs on every visible row in the table.

6. Table and detail panel isolated into their own fragment

python

```
@st.fragment
def render_alerts_table_and_details(filtered_alerts, use_12h
r):
    ...

@st.fragment
def render_alerts_analytics(filtered_alerts):
    ...
```

Selecting a row or changing the page/sort no longer forces the three Plotly analytics charts to redraw — they only redraw when the fragment they live in actually reruns for their own reasons. This is what made row-clicks feel instant instead of dragging chart re-rendering along with them.

7. Pagination on both Alerts and Traffic tables

python

```
def _paginate(df, state_key_prefix, default_rows=25):  
    ...
```

Caps how many rows `st.dataframe()` ever has to render at once (25/50/100/250/All, selectable), instead of dumping the full result set into the grid on every rerun.

8. Charts switched from Streamlit's built-in charts to Plotly

`severity_breakdown_chart`, `alerts_over_time_chart`, `top_signatures_chart`, `traffic_over_time_chart`, `top_talkers_chart`, `protocol_distribution_chart` all now use `plotly.express` with a fixed, locked-down config (`displayModeBar: False`, `scrollZoom: False`, `dragmode: False`) instead of `st.bar_chart` / `st.line_chart`. This is primarily a visual upgrade (labeled bars, consistent blue theme, dynamic axis headroom) — not a speed change per se, though disabling zoom/pan interactivity does mean less JS work in the browser per chart.

9. Adjustable DB fetch limit

python

```
fetch_limit = st.sidebar.select_slider(  
    "Max DB Fetch Limit",  
    options=[250, 500, 1000, 2000],  
    value=500,  
    ...  
)
```

Lets you directly control how many rows get pulled from SQLite into memory in the first place, on top of the pagination that controls how many get *rendered*.